

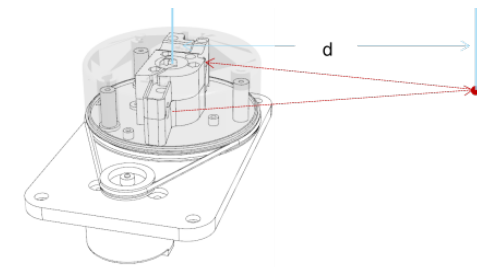
SY31 – TP #4

Détection d'Objets au LiDAR

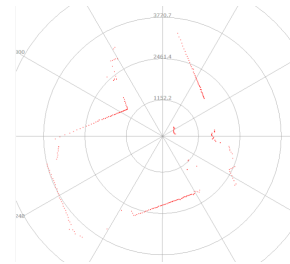
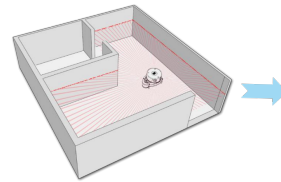
P26

1 Introduction

L'objectif de ce TP est de détecter des objets fixes ou mobiles en utilisant le LiDAR du Turtlebot. Le LiDAR (Light Detection And Ranging) est un télémètre laser. Nous utiliserons dans ce TP un télémètre à balayage permettant de mesurer des distances sur différents angles. Les distances peuvent être mesurées de plusieurs façons, par exemple: par triangulation, par calcul du temps de vol d'un signal laser ou par calcul du déphasage entre l'onde émise et l'onde reçue. Le LiDAR permet alors d'obtenir un scan de l'environnement du robot pour un plan donné.



(a) Calcul de distance par triangulation



(b) Scan LiDAR de l'environnement

Nous allons travailler sur un nœud ROS qui prendra en entrée les points 2D mesurés par le LiDAR et sortira une représentation de l'environnement. Les données LiDAR, de type `sensor_msgs/msg/LaserScan` sont sur le topic `/scan`.

Ces données ainsi que les autres sorties de nos nœuds sont visualisables dans `rviz2`.

2 Pré-Traitement du Nuage de Points

Nous allons dans un premier temps traiter les données LiDAR pour les rendre exploitables. En effet, le capteur retourne des coordonnées polaires à partir desquelles il est difficile de déterminer des formes. Nous allons donc transformer ces données en un *nuage de points 2D*, c'est à dire une liste de coordonnées cartésiennes 2D. De plus, certains points peuvent se refléter sur le robot lui-même, ce qui entraînera des mesures incohérentes. On souhaite donc les retirer avant de continuer.

Exercice 1 : Créer un workspace, un package `lidar` et y mettre les nœud récupérés sur Moodle. Compléter la fonction `callback` de `transformer` afin de retirer les points à moins de 10 cm du robot et de convertir les points en coordonnées cartésiennes avec l'équation suivante :

$$\begin{cases} x = r * \cos \theta \\ y = r * \sin \theta \end{cases}$$

Vérifier dans `rviz2` que les points générés correspondent bien à ceux passant sur `/scan`.

⚠ Warning

Penser à mettre `base_scan` à la place de `map` dans *Fixed Frame* en haut à gauche. Il est également possible de sauvegarder la configuration de RViz et de la re-charger en faisant `rviz -d VOTRE_FICHER`.

3 Filtrage par Intensité

En plus de mesurer la distances à des obstacles, l'intensité de la lumière réfléchie puis reçue permet d'inférer la réflectivité des matériaux. Dans cette partie, nous allons utiliser la mesure d'*intensité* fournie par le LiDAR pour isoler les points les plus réfléchifs¹.

Dans un premier temps nous allons déterminer un seuil de filtrage. Pour cela, montrez un objet réfléchif devant le LiDAR du robot. Dans RViz, utilisez l'outil de sélection (1) pour sélectionner les points correspondants à cet objet et regarder dans le panneau de sélection (2) l'intensité qui leur correspond.

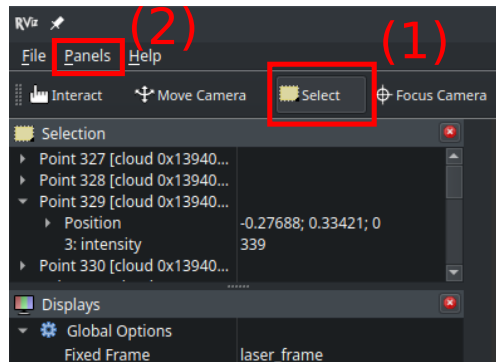


Figure 2: Sélection de points dans RViz.

Ensuite, récupérez et complétez le fichier `intensity_filter.py` afin de ne publier que les points dépassant le seuil déterminé précédemment. Vous pourrez vous servir de l'attribut `intensity_threshold` qui peut être mis-à-jour sans relancer le nœud grâce à la commande

```
ros2 param set /intensity_filter intensity_threshold NOUVELLE_VALEUR
```

¹Pour aller plus loin, l'intensité fournie par le capteur n'est pas vraiment une mesure de réflectivité. Le lien entre intensité et réflectivité dépend également de la distance de mesure, de l'angle d'incidence avec l'obstacle et du critère utilisé par le capteur pour seuiliser la mesure. Il faudrait donc réaliser une calibration avec des matériaux de réflectivité connue selon tous ces critères, procédure que nous ne suivrons pas dans cette UV.

4 Clustering

Nous allons ensuite regrouper les points afin de détecter des objets distincts. Nous utiliserons pour cela un identifiant de cluster $C[i]$ associé à chaque point $P[i]$. Tous les points $P[i]$ ayant le même $C[i]$ sont considérés comme regroupés et $C[i] = 0$ signifie l'absence de cluster.

Le pseudo-code suivant présente le fonctionnement de l'algorithme que nous allons utiliser :

Algorithm 1 Segmentation de point

```
 $C[i] \leftarrow 0 \quad \forall i$ 
for  $i \in \{0 \dots n_{pt}\}$  do
  for  $j \in \{1 \dots k\}$  do
     $d[j-1] \leftarrow ||P[i] - P[i-j]||$ 
  end for
   $d_{min} \leftarrow \min(d)$ 
   $j_{min} \leftarrow \operatorname{argmin}(d) + 1$ 

  if  $d_{min} < D$  then
    if  $C[i - j_{min}] == 0$  then
       $C[i - j_{min}] \leftarrow \max(C) + 1$ 
    end if
     $C[i] \leftarrow C[i - j_{min}]$ 
  end if
end for
return  $C$ 
```

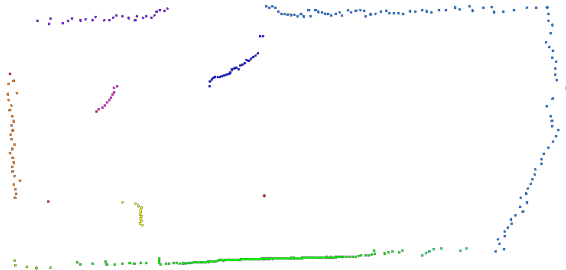


Figure 3: Clusters de points LiDAR

Exercice 2 : Dédurre à partir du pseudo code l'utilité des constantes k et D , puis proposer des valeurs pour ces paramètres.

Exercice 3 : Quel problème peut-on rencontrer à utiliser un D de valeur fixe ? Proposer une solution pour régler ce problème.

Exercice 4 : Implémenter ce pseudo-code dans le nœud `clusterer` et visualiser les clusters de points LiDAR dans `rviz2`. Pour colorier les points en fonction de leur cluster, il faut changer le *Channel Name* en `c`.

Exercice 5 : Lancer plusieurs nœuds en parallèle à la main avec `ros2 run` est rapidement fastidieux. Il existe un outil pour automatiser les `ros2 run`: `ros2 launch`. Il s'agit d'un fichier XML décrivant les nœuds à lancer. Étudier la documentation pour automatiser la transformation et le clustering.

Warning

Par défaut, les nœuds lancés avec `ros2 launch` n'afficheront rien dans la console. Pour les y forcer, on peut ajouter l'option `output="screen"` à la balise `node`.

5 Détection de Formes

5.1 Cylindre englobant

Afin d'éviter que le robot n'entre en collision avec des obstacles sans pour autant avoir à déterminer une distance avec chaque point LiDAR, nous pouvons représenter chaque cluster par un cylindre englobant. Cette détection de forme est assez simple à déterminer en calculant le centre et le rayon d'un cercle de hauteur infinie à partir des points du cluster. Ainsi, en modélisant également le robot par un centre et un rayon fixe, une collision avec l'obstacle sera évitée si la distance du centre du robot au centre de l'obstacle est supérieure à la somme des deux rayons et des incertitudes de la position du robot et de la mesure LiDAR:

$$\|c_{\text{robot}} - c_{\text{cluster}}\| > r_{\text{robot}} + r_{\text{cluster}} + 3\sigma_{\text{robot}} + 3\sigma_{\text{LiDAR}}$$

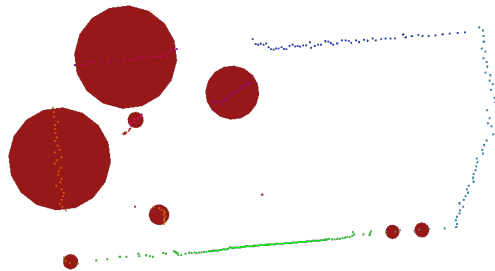


Figure 4: Cylindres englobants des clusters.

Exercice 6 : Implémenter cette méthode dans le nœud `shaper_cylinder` et visualiser les cylindres de clusters dans `rviz2`.

5.2 Boite englobante

La représentation par cylindre est simple mais pessimiste. Afin de raffiner notre représentation des obstacles, nous pouvons utiliser des boîtes englobantes. Pour cela, on calcule le centre, la longueur et la largeur de chaque cluster.

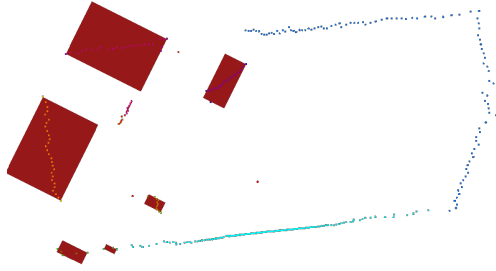


Figure 5: Boites englobantes des clusters

Exercice 7 : Implémenter cette méthode dans le nœud `shaper_bbox` et visualiser les boites englobantes de clusters dans `rviz2`.

i Note

On peut remarquer que les boîtes sont encore trop pessimistes car non orientées avec le cluster. Il est possible d'estimer l'orientation du cluster afin d'affiner la représentation mais cela dépasse le cadre de ce TP.

6 Amélioration du clustering

Vous aurez remarqué que vos boites englobantes sont parfois trop petites ou trop grandes. Cela est dû au fait que le clustering n'impose pas de limite de nombre de points.

Exercice 8 : Ajouter à `clusterer.py` une limite minimale et maximale à la taille des clusters. Vérifiez dans un RViz que vos modification ont l'effet escompté.

Finalement, l'algorithme de clustering proposé dans ce TP implique une discontinuité de cluster entre le premier et le dernier point.

Exercice 9 : Modifier l'algorithme de `clusterer.py` pour éliminer cette discontinuité.

7 Bonus - Détection de polygones

En plus des cylindres et boites englobantes, il est possible de représenter un obstacle encore plus finement en déterminant une polygone, une suite de segments, passant par les points du cluster. Afin d'alléger l'obstacle, cette polygone ne prend pas en compte l'ensemble des points LiDAR mais est simplifiée en utilisant l'algorithme de Ramer-Douglas-Peucker, Alg. 2, aussi appelé algorithme de la corde, illustré dans la figure suivante :

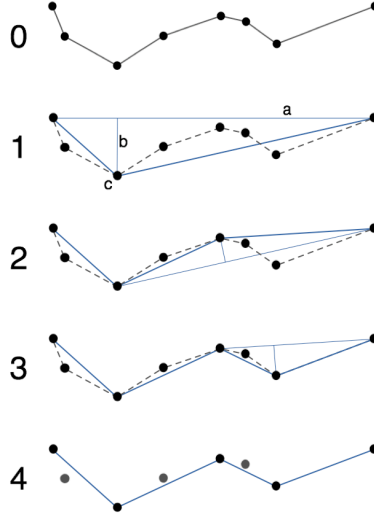


Figure 6: Algorithme de Ramer-Douglas-Peucker permettant de réduire le nombre de points d'une polygline

L'algorithme de Ramer-Douglas-Peucker est défini par récurrence en déterminant à chaque itération le point $P[j_{max}]$ le plus éloigné de la corde $[P[0], P[n_{pt}]]$ (le segment allant du premier au dernier point de l'ensemble des points). L'ensemble des points est alors divisé en deux sous-ensembles séparés au niveau de l'indice j_{max} sur lesquels l'algorithme est alors appliqué. L'algorithme s'arrête lorsque la distance maximale d_{max} est inférieure à un seuil ϵ . Les points des extrémités des ensembles sont concaténés pour obtenir la polygline :

Algorithm 2 Ramer-Douglas-Peucker.

```

Initialiser la distance max  $d_{max}$  et sont index  $j_{max}$  à 0
for all  $i \in \{1 \dots n_{pt} - 1\}$  do
    Calculer la distance  $d[i]$  entre le point  $P[i]$  et le segment  $[P[0], P[n_{pt}]]$ 
end for
 $[d_{max}, j_{max}] = \max(d)$ 
if  $d_{max} > \epsilon$  ** ** then
     $P_{res1} = \text{rdp}(\{P[i], i \in \{0 \dots j_{max}\}\}, \epsilon)$ 
     $P_{res2} = \text{rdp}(\{P[i], i \in \{j_{max} \dots n_{pt}\}\}, \epsilon)$ 
     $P_{res} = \{P_{res1}[0 \dots n_{pt1} - 1], P_{res2}[0 \dots n_{pt2}]\}$ 
else
     $P_{res} = \{P[0], P[n_{pt}]\}$ 
end if
return  $P_{res}$ 

```

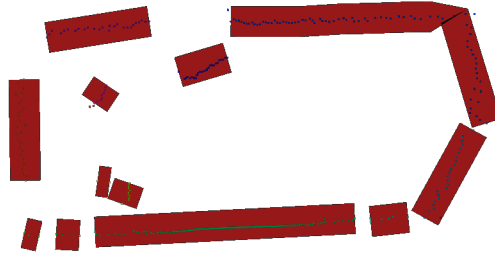


Figure 7: Polylines des Clusters de points LiDAR

Exercice 10 : Implémenter cette méthode dans le nœud `shaper_polyline.py` et visualiser les polygones de clusters dans RViz.